Command-Line Interface (CLI) Reference

The `sfnext` CLI is your primary tool for developing and deploying Storefront Next projects. It's provided by the `@salesforce/storefront-next-dev` package, which is installed as a project dependency–not globally.

Running CLI Commands

Using Project package.json Scripts

Storefront Next projects include pre-configured scripts in `package.json` for common tasks:

```
pnpm dev        # Start the development server
pnpm build      # Create a production build
pnpm start      # Preview the production build locally
pnpm push       # Deploy to Managed Runtime
```

Running Commands Directly

For commands that don't have a `package.json` script, or when you need to pass specific options, run `pnpm sfnext` from your project's root directory:

```
pnpm sfnext <command> [options]
```

For example:

```
pnpm sfnext dev --port 3000
pnpm sfnext extensions list
```

You can also view all available commands and options:

```
pnpm sfnext --help
pnpm sfnext <command> --help
```

This works because `pnpm` automatically resolves binaries installed in your project's `node_modules/.bin`. Running `sfnext` this way

Contents

> Don't install `@salesforce/storefront-next-dev` globally. Always run commands from within your project directory so that the CLI version stays in sync with your dependencies.

Creating a Storefront Without a Project

The `create-storefront` command is the one exception–it runs *before* a project exists, so there's no local `node_modules` to resolve from. For this command, use `pnpm dlx` to download and execute the package directly:

```
1  pnpm dlx @salesforce/storefront-next-dev create-storef
```

For all other commands, use `pnpm sfnext` from within your project.

Commands

Command	package.json Script	Description
`create-storefront`	–	Scaffold a new storefront project from a template.
`dev`	`pnpm dev`	Start the development server with hot module replacement.
`preview`	`pnpm start`	Preview a production build locally.
`push`	`pnpm push`	Build a bundle and deploy it to Managed Runtime.
`create-bundle`	–	Create a deployment bundle without pushing.
`extensions`	–	Manage feature extensions (install, remove, create, list).

create-storefront

Scaffold a new Storefront Next project. The command clones a template repository, walks you through extension selection, and configures your environment variables.

Because this command creates a new project, it's the one CLI command you run *before* a project exists. Use `pnpm dlx` to download and run it without a prior installation:

Options

Option	Description
`-n, --name <name>`	Name for the storefront. Skips the interactive prompt.
`-t, --template <template>`	Template URL or local path (for example, a GitHub URL or `file:///path/to/template`).
`-l, --local-packages-dir <dir>`	Local monorepo packages directory. Used with `file://` templates to pre-fill dependency paths.
`-v, --verbose`	Enable verbose output.

Examples

Create a storefront interactively:

```
pnpm dlx @salesforce/storefront-next-dev create-storef
```

Create a storefront with a specific name:

```
pnpm dlx @salesforce/storefront-next-dev create-storef
```

After the command completes, follow the on-screen instructions to install dependencies and start developing:

```
cd my-storefront
pnpm install
pnpm build
pnpm dev
```

For a walkthrough of creating a storefront with the CLI, see Explore Storefront Next Code Locally.

dev

Start a local development server powered by Vite with full server-side rendering (SSR) support. The server includes hot module replacement (HMR) for both client and server code, and an API proxy to B2C Commerce.

```
pnpm sfnext dev [options]
```

Options

Option	Description	Default
`-d, --project-directory <dir>`	Path to the project directory.	Current directory
`-p, --port <port>`	Port number for the dev server.	`5173`

Examples

Start the dev server:

```
pnpm dev
```

Start the dev server on a custom port:

```
pnpm sfnext dev --port 3000
```

Start with the Node.js debugger attached:

```
pnpm dev:debug
```

What the Dev Server Does

- Runs Vite in middleware mode within an Express server for full SSR support.
- Proxies B2C Commerce API requests through `/mobify/proxy/api` so that you can develop against live data without CORS issues.
- Loads environment variables from your `.env` file.
- Prints your Commerce API configuration (short code, organization ID, site ID) at startup.

preview / start

Start a local server that serves your production build. Use this to test production behavior before deploying. If no build exists, the command automatically runs `pnpm build` first.

```
pnpm start
```

Or with options:

```
pnpm sfnext preview [options]
```

Options

Option	Description	Default
`-d, --project-directory <dir>`	Path to the project directory.	Current directory
`-p, --port <port>`	Port number for the preview server.	`3000`

Examples

Preview the production build:

```
pnpm start
```

Preview on a custom port:

```
pnpm sfnext preview --port 8080
```

How Preview Differs from Dev

Feature	`dev`	`preview`
Build type	On-the-fly via Vite	Pre-built production bundle
HMR	Yes	No
Static asset serving	Vite handles	Express with compression
Performance	Optimized for speed of iteration	Reflects production behavior

push

```
pnpm push
```

Or with options:

```
pnpm sfnext push [options]
```

Options

Option	Description	Default
`--project-directory <dir>`	Path to the project directory.	Current directory
`-b, --build-directory <dir>`	Path to the build output directory.	Auto-detected
`-m, --message <message>`	Descriptive message for the bundle.	`git branch:commit`
`-p, --project <slug>`	Unique project identifier on Managed Runtime.	From `SFCC_MRT_PROJECT` or `MRT_PROJECT` env var
`-e, --environment <target>`	Target environment to deploy to.	From `SFCC_MRT_ENVIRONMENT` or `MRT_TARGET` env var
`-w, --wait`	Wait for the deployment to finish before exiting.	`false`
`--api-key <key>`	API key for MRT authentication.	From `SFCC_MRT_API_KEY` env var
`--credentials-file <file>`	Path to the MRT credentials file.	From `MRT_CREDENTIALS_FILE` env var
`--cloud-origin <origin>`	MRT API origin URL.	Production default

	config file.	
`-i, --instance <name>`	Named instance from config file.	From `SFCC_INSTANCE` env var

Authentication

Authenticate with Managed Runtime using one of these methods:

- **Credentials file** (default): The CLI reads stored credentials automatically. Use `--credentials-file` to specify a custom path.
- **API key**: Pass `--api-key` on the command line or set the `SFCC_MRT_API_KEY` environment variable.

Setting Up MRT Resources

Before you can deploy, you need a Managed Runtime project and SLAS client credentials. Use the B2C Developer Tooling CLI to create these resources.

Examples

Build and deploy to MRT:

```
pnpm build
pnpm push
```

Deploy to a specific environment and wait for completion:

```
pnpm sfnext push --environment staging --wait
```

Deploy with an API key:

```
pnpm sfnext push --api-key your-api-key
```

create-bundle

Create a deployment bundle and save it to disk without pushing to Managed Runtime. This is useful for inspecting bundle contents or integrating with custom deployment pipelines.

```
pnpm sfnext create-bundle -d <project-directory> [opti
```

`-d, --project-directory <dir>`	**(Required)** Path to the project directory.	
`-b, --build-directory <dir>`	Path to the build output directory.	Auto-detected
`-o, --output-directory <dir>`	Directory where bundle files are written.	`.bundle`
`-m, --message <message>`	Descriptive message for the bundle.	`git branch:commit`
`-s, --project-slug <slug>`	Unique project identifier on Managed Runtime.	From `.env` `MRT_PROJECT` or `package.json` name

Output

The command generates two files in the output directory:

- `bundle.tgz` – The compressed deployment bundle.
- `bundle.json` – Bundle metadata (message, SSR parameters, file sizes).

Example

```
pnpm build
pnpm sfnext create-bundle -d . -o ./my-bundle
```

extensions

Manage feature extensions for your storefront project. Extensions add pre-built functionality, such as store locators or theme switchers.

```
pnpm sfnext extensions <subcommand> [options]
```

extensions list

List all extensions currently installed in your project.

`-d, --project-directory <dir>`	Path to the project directory.	Current directory

Example

```
pnpm sfnext extensions list
```

extensions install

Install an extension from a source template repository. The CLI clones the source repository, lets you select an extension, and copies the extension files into your project.

```
pnpm sfnext extensions install [options]
```

Option	Description	Default
`-d, --project-directory <dir>`	Path to the target project directory.	Current directory
`-e, --extension <extension>`	Extension marker value to install (for example, `SFDC_EXT_STORE_LOCATOR`). Skips the interactive prompt.	
`-s, --source-git-url <url>`	Git URL of the source template that contains extensions.	Default Storefront Next template
`-v, --verbose`	Enable verbose output.	

If the extension has dependencies on other extensions, the CLI detects them and installs the full dependency chain in the correct order.

Examples

Install an extension interactively:

```
pnpm sfnext extensions install
```

Install a specific extension by marker:

extensions remove

Remove one or more installed extensions from your project. The CLI automatically detects and removes dependent extensions.

```
pnpm sfnext extensions remove [options]
```

Option	Description
`-d, --project-directory <dir>`	Path to the project directory.
`-e, --extensions <extensions>`	Comma-separated list of extension markers to remov example, `SFDC_EXT_STORE_LOCATOR,SFDC_EXT_THEME_SWI` Skips the interactive prompt.
`-v, --verbose`	Enable verbose output.

Example

Remove extensions interactively:

```
pnpm sfnext extensions remove
```

Remove specific extensions by marker:

```
pnpm sfnext extensions remove -e SFDC_EXT_STORE_LOCATO
```

extensions create

Scaffold a new extension in your project. This command creates the extension directory structure with placeholder folders for components, hooks, locales, and routes.

```
pnpm sfnext extensions create [options]
```

`-n, --name <name>`	Name of the extension (for example, `"My Extension"`).
`-d, --description <description>`	Description of the extension.

Example

```
pnpm sfnext extensions create --name "Store Locator" -
```

The generated directory structure:

```
src/extensions/store-locator/
  components/
  hooks/
  locales/
  routes/
  README.md
```

Environment Variables

Several CLI commands read configuration from environment variables defined in your `.env` file.

Variable	Used By
`SFCC_MRT_PROJECT`	`push`, `create-bundle`
`SFCC_MRT_ENVIRONMENT`	`push`
`SFCC_MRT_API_KEY`	`push`
`SFCC_PROJECT_DIRECTORY`	All commands
`MRT_PROJECT`	`push`, `create-bundle`

`PUBLIC__app__commerce__api__shortCode`	`dev`, `preview`
`PUBLIC__app__commerce__api__organizationId`	`dev`, `preview`
`PUBLIC__app__commerce__api__clientId`	`dev`, `preview`
`PUBLIC__app__commerce__api__siteId`	`dev`, `preview`

See Also

- Build Tools – Vite configuration and build system details.
- Project Configuration – Environment variables and storefront configuration.
- B2C Developer Tooling CLI: Storefront Next – Create SLAS clients and Managed Runtime resources.